SelVeri LP: Logic Programming Extension for SelVeri Syntax and Semantics Document

Yusuf Demir
Yağız Kaan Aydoğdu

June 12, 2026

Abstract

This document formalizes `SelVeri LP`, a constraint-driven logic programming extension to the `SelVeri` language. By blurring the line between executable imperatives and static verification, `SelVeri LP` allows programmers to utilize First-Order Logic (FOL) and Past-Linear-Temporal Logic (pLTL) specifications directly as boolean truth values in control flow, and to dynamically extract satisfying witnesses from the `Z3` SMT solver directly into the program state.

1 Extended High-Level Syntax

To support the logic programming capabilities of `SelVeri LP`, we extend the core grammar of `SelVeri`. We introduce a mechanism to evaluate specifications as standard boolean expressions, and a new arithmetic (logic) expression to query the solver for existential witnesses.

1.1 Boolean Expressions

We extend the $\langle BExp \rangle$ ruleset. A specification enclosed in curly braces can now be utilized anywhere a boolean expression is expected (e.g., in `if` guards or `while` conditions).

$$\langle BAtom \rangle \quad ::= \quad \texttt{true} \mid \texttt{false} \mid \langle Comparison \rangle \mid \langle AExp \rangle \mid (\langle BExp \rangle) \mid \{\langle Spec \rangle\}$$

1.2 Arithmetic Expressions

We extend the $\langle AAtom \rangle$ ruleset to include the `obtain` expression. This expression queries the solver for a witness and evaluates to its value.

$$\begin{aligned} \langle AAtom \rangle \quad ::= \quad & \langle Imm \rangle \mid \langle Ident \rangle \mid \texttt{len}(\langle Ident \rangle) \mid \langle IndexedValue \rangle \mid (\langle AExp \rangle) \\ & \mid \texttt{read}() \mid \texttt{obtain}(\&\langle Ident \rangle, \langle Spec \rangle) \end{aligned}$$

Because `obtain` is an arithmetic expression, it can appear anywhere a value is expected: in assignments (`y := obtain(&x, ϕ)`), as an argument to I/O functions (`writeline(obtain(&x, ϕ))`), inside arithmetic operations (`obtain(...) + 1`), and so on.

2 Extended High-Level Semantics

2.1 Semantics of Specification-based Booleans

Let $\mathcal{M}_{\texttt{Z3}}$ be the mapping function that outputs the corresponding Z3 solver state for a given program state σ and scope s, and $\mathcal{ZF}$ be the FOL mapping function. The boolean valuation function $\mathcal{B}$ is extended for specifications as follows:

$$\mathcal{B}(\{\phi\}, \sigma, s) \quad = \begin{cases} 1 & \text{if } \mathcal{M}_{\texttt{Z3}}(\sigma, s) \models \mathcal{ZF}(\phi) \\ 0 & \text{if } \mathcal{M}_{\texttt{Z3}}(\sigma, s) \not\models \mathcal{ZF}(\phi) \\ \bot & \text{if Z3 returns } \texttt{unknown} \text{ or timeout} \end{cases}$$

2.2 Semantics of `obtain`

We define an extraction function $\mathcal{W}(x, \phi, \sigma, s)$. If $\mathcal{M}_{\texttt{Z3}}(\sigma, s) \models \mathcal{ZF}(\phi)$, `Z3` produces a satisfying model M. $\mathcal{W}$ evaluates the existentially bound variable x under this model M and returns its concrete value $v \in \textbf{Val}$. If the formula is unsatisfiable or undecidable, $\mathcal{W}$ yields $\bot$.

Because `obtain` is an arithmetic expression, its semantics extend the arithmetic valuation function $\mathcal{A}$:

$$\mathcal{A}(\texttt{obtain}(\&x, \phi), \sigma, s) \quad = \begin{cases} v & \text{if } \mathcal{W}(x, \phi, \sigma, s) = v \neq \bot \\ \bot & \text{if } \mathcal{W}(x, \phi, \sigma, s) = \bot \end{cases}$$

Table 1: Extension of $\mathcal{A}$ for witness extraction

When $\mathcal{A}$ yields $\perp$, the standard error propagation rule of the base language applies: any statement evaluating an arithmetic expression that produces $\perp$ transitions to the erroneous state $\hat{\sigma}$. Otherwise, the expression evaluates to the concrete witness value v, which can be used directly in any arithmetic context. For example:

```
y := obtain(&x, Exists x in Int . &x = 5);
writeline(obtain(&w, Exists w in Int . &w = 42));
```

2.3 Accepted Forms of `obtain`

The concrete syntax is `obtain(&`x`,` ϕ`)` where `&`x names the target bound variable whose witness is extracted, and ϕ is a specification containing an existential quantifier binding x. The specification ϕ must contain a sub-formula of the form $\exists\, x \in D.\ \psi$ for some domain D and body ψ.

Supported Domains

The domain D of the existential quantifier determines both the Z3 sort of the bound variable and the constraints added to the solver:

Domain form	**Z3 sort**	**Constraint added**
`Int`	$\mathbb{Z}$ (unbounded)	none
`Real`	$\mathbb{R}$ (unbounded)	none
`[`l`...`h`]` (range)	$\mathbb{Z}$	$l \le x \le h$
`[`l`;`h`]`, `(`l`;`h`)`, etc. (interval)	$\mathbb{R}$	corresponding endpoint inclusion
`[`$v_1, v_2, \ldots, v_n$`]` (value list)	$\mathbb{Z}$	$x = v_1 \lor x = v_2 \lor \cdots \lor x = v_n$

Nested Quantifiers

The specification may contain multiple nested quantifiers. When `obtain` targets a specific bound variable, all quantifiers on the path from the specification root to the target existential are *flattened*: each bound variable is introduced as a free Z3 variable with its domain constraints, and the innermost body is checked for satisfiability. For example:

$$\texttt{obtain(\&}b,\ \ \exists\, a \in [1 \ldots 10].\ \exists\, b \in [5, 6, 7, 9].\ \texttt{\&}a * \texttt{\&}a = \texttt{\&}b)$$

Both a and b are introduced as free integer variables. The solver finds a satisfying assignment (e.g., $a = 3$, $b = 9$) and the value of the *target* variable b is returned as the value of the expression. Note that the choice of witness is non-deterministic when multiple satisfying assignments exist, as established in the determinism analysis below.

3 Determinism analysis for SelVeri LP

The original `SelVeri` language was strictly deterministic, meaning its small-step operational semantics transition relation ($\rightarrow$) formed a partial function. Introducing the `SelVeri LP` constraint-driven extensions requires us to re-evaluate this property. We must analyze the two new additions independently: specification-based boolean expressions ($\{\phi\}$) and witness extraction (`obtain`).

3.1 Determinism of Specification-Booleans

Theorem: A subset of `SelVeri LP` that extends `SelVeri` *only* with the specification-boolean expression $\{\phi\}$ remains strictly deterministic.

Proof. To prove this, we must show that the extended boolean valuation function $\mathcal{B}$ still maps any valid configuration to exactly one value in $\{0, 1, \perp\}$.

For any given specification $\{\phi\}$, the function evaluates the entailment $\mathcal{M}_{\texttt{Z3}}(\sigma, s) \models \mathcal{ZF}(\phi)$. For a fully instantiated state and scope, this is a closed First-Order Logic formula. The semantic truth value of a closed FOL formula is absolute—it is either satisfiable (1), unsatisfiable (0), or undecidable/timeout ($\perp$). Assuming a standardized solver oracle (`Z3`), this evaluation yields exactly one deterministic outcome.

Because the evaluation of $\{\phi\}$remains a partial function, the mutual exclusivity of the operational semantic rules relying on $\mathcal{B}$ (such as $[\text{if}^1_{\text{os}}]$ and $[\text{if}^0_{\text{os}}]$) is perfectly preserved. Thus, routing control flow via specifications maintains the deterministic guarantees of the language. □

3.2 Non-Determinism of Witness Extraction

Theorem: The introduction of the `obtain` expression renders the `SelVeri LP` language semantically non-deterministic.

Proof. To prove non-determinism, we must show that there exists a configuration $\langle S, \sigma, s\rangle$ that can validly transition to more than one distinct subsequent configuration.

Consider the statement $S = y$ `:= obtain(&`x, $\exists\, x \in \textbf{Int}.\ \&x > 0 \land \&x < 5$) evaluated under a standard integer scope where y is declared. According to the operational semantic rule $[\text{obt}_{\text{os}}]$, the `obtain` expression evaluates to a concrete witness v such that the model satisfies the constraints.

In this scenario, the constraints are satisfied by multiple distinct models, specifically where $x \in \{1, 2, 3, 4\}$. Under the formal semantics of the extraction function $\mathcal{W}$, any of these witnesses is a mathematically valid yield. Consequently, the single initial configuration can legally transition to any of the following distinct states:

- $\langle \sigma[y \mapsto 1],\ s\rangle$
- $\langle \sigma[y \mapsto 2],\ s\rangle$
- $\langle \sigma[y \mapsto 3],\ s\rangle$
- $\langle \sigma[y \mapsto 4],\ s\rangle$

Because the transition relation ($\rightarrow$) maps a single pre-state to a set of multiple valid post-states, it ceases to be a partial function and becomes a one-to-many relation. While a specific underlying implementation of the `Z3` solver may deterministically return the same witness (e.g., always returning 1) due to internal heuristics, the *formal semantics* of the language place no such restriction. Therefore, `SelVeri LP` is inherently non-deterministic. □

4 SelVerIR LP: Extended Intermediate Representation

To execute these high-level paradigms on the abstract machine, we introduce two new `SelVerIR` instructions: `VERIP` and `OBT`.

4.1 IR Syntax Extensions

The instruction set $\langle Command \rangle$ is extended as follows:

$$\langle Command \rangle \quad ::= \quad \ldots \mid \texttt{VERIP}\ \langle IntLit \rangle, \langle Spec \rangle \mid \texttt{OBT}\ \langle ident \rangle, \langle IntLit \rangle, \langle Spec \rangle$$

4.2 IR Operational Semantics

$\langle \texttt{VERIP}\ id,\ \phi : c,\ e,\ \sigma,\ s,\ pc \rangle$	$\triangleright$	$\begin{cases} \langle c,\ 1 : e,\ \sigma,\ s,\ pc+1 \rangle & \text{if } \mathcal{M}_{\texttt{Z3}}(\sigma, s) \models \mathcal{ZF}(\phi) \\ \langle c,\ 0 : e,\ \sigma,\ s,\ pc+1 \rangle & \text{if } \mathcal{M}_{\texttt{Z3}}(\sigma, s) \not\models \mathcal{ZF}(\phi) \\ \langle c,\ e,\ \hat{\sigma},\ s,\ pc+1 \rangle & \text{if Z3 returns } \texttt{unknown} \end{cases}$
$\langle \texttt{OBT}\ x,\ id,\ \phi : c,\ e,\ \sigma,\ s,\ pc \rangle$	$\triangleright$	$\begin{cases} \langle c,\ v : e,\ \sigma,\ s,\ pc+1 \rangle & \text{if } \mathcal{W}(x, \phi, \sigma, s) = v \neq \bot \\ \langle c,\ e,\ \hat{\sigma},\ s,\ pc+1 \rangle & \text{if } \mathcal{W}(x, \phi, \sigma, s) = \bot \end{cases}$

Table 2: Operational semantics of SelVerIR LP instructions

The `VERIP` instruction mimics the original `VERI` instruction, but instead of asserting the specification and halting the program on failure, it safely pushes the boolean truth value (1 or 0) to the stack e. This allows the abstract machine to consume the logical result for conditional jumps (e.g., `JZ`).

The `OBT` instruction queries the solver for the identifier x under the constraint ϕ. If a valid witness v is synthesized, it is pushed onto the evaluation stack e. This mirrors the semantics of other value-producing IR instructions such as `PUSH` and `READ`.

5 Compilation to SelVerIR LP

With the operational semantics of the abstract machine defined, we seamlessly extend the compilation function $\mathcal{C}$ and the boolean compilation helper $\mathcal{CB}$. Because `obtain` is now an arithmetic expression, its compilation rule belongs to the arithmetic compilation function $\mathcal{CA}$.

$$\mathcal{CB}(\{\phi\}) = \texttt{VERIP}\ id_\phi, \phi$$

$$\mathcal{CA}(\texttt{obtain}(\&x, \phi)) = \texttt{OBT}\ x,\ id_\phi,\ \phi$$

Table 3: Compilation rules for SelVeri LP extensions

6 Provably Correct Implementation of SelVeri LP

To ensure that the `SelVeri LP` extension preserves the rigorous mathematical guarantees of the original `SelVeri` language, we must extend the compiler correctness proofs. Specifically, we must demonstrate that the compilation mapping of the new boolean specification expression and the `obtain` expression maintain strict equivalence between the high-level semantics and the `SelVerIR LP` abstract machine.

6.1 Correctness of Specification as Boolean Expressions

Theorem: Given any non-erroneous program state σ, scope s, and boolean specification expression $b = \{\phi\}$, the compilation $\mathcal{CB}$ is correct:

$$\langle \mathcal{CB}(\{\phi\}) : c,\, e,\, \sigma,\, s,\, pc \rangle \;\triangleright^*\; \langle c,\, \mathcal{B}(\{\phi\}, \sigma, s) : e,\, \sigma,\, s,\, pc + 1 \rangle$$

Proof. We prove this by evaluating the definition of $\mathcal{CB}(\{\phi\})$ alongside the operational semantics of the `VERIP` instruction.

$$\langle \mathcal{CB}(\{\phi\}) : c,\, e,\, \sigma,\, s,\, pc \rangle = \langle \texttt{VERIP}\ id_\phi, \phi : c,\, e,\, \sigma,\, s,\, pc \rangle \quad \text{(definition of } \mathcal{CB})$$

We consider the two non-erroneous subcases based on the satisfiability of the formula ϕ:

- **Subcase 1:** $\mathcal{M}_{\texttt{Z3}}(\sigma, s) \models \mathcal{ZF}(\phi)$

$$\begin{aligned} \langle \texttt{VERIP}\ id_\phi, \phi : c,\, e,\, \sigma,\, s,\, pc \rangle &\triangleright \langle c,\, 1 : e,\, \sigma,\, s,\, pc + 1 \rangle && \text{(operational semantics of } \texttt{VERIP}) \\ &= \langle c,\, \mathcal{B}(\{\phi\}, \sigma, s) : e,\, \sigma,\, s,\, pc + 1 \rangle && \text{(definition of } \mathcal{B}) \end{aligned}$$

- **Subcase 2:** $\mathcal{M}_{\texttt{Z3}}(\sigma, s) \not\models \mathcal{ZF}(\phi)$

$$\begin{aligned} \langle \texttt{VERIP}\ id_\phi, \phi : c,\, e,\, \sigma,\, s,\, pc \rangle &\triangleright \langle c,\, 0 : e,\, \sigma,\, s,\, pc + 1 \rangle && \text{(operational semantics of } \texttt{VERIP}) \\ &= \langle c,\, \mathcal{B}(\{\phi\}, \sigma, s) : e,\, \sigma,\, s,\, pc + 1 \rangle && \text{(definition of } \mathcal{B}) \end{aligned}$$

In the edge case where `Z3` returns `unknown` (or times out), the high-level evaluation yields $\perp$, which deterministically routes to an error state. Mirroring this perfectly, the IR instruction transitions to $\hat{\sigma}$. Thus, the equivalence strictly holds for all evaluations. □

6.2 Correctness of Witness Extraction (`obtain`)

Theorem: For any valid arithmetic expression $E = \texttt{obtain}(\&x, \phi)$, its compilation is correct. Assuming the witness extraction succeeds ($\mathcal{W}(x, \phi, \sigma, s) = v \neq \perp$):

$$\mathcal{A}(E, \sigma, s) = v$$

implies:

$$\langle \mathcal{CA}(E) : c,\, e,\, \sigma,\, s,\, pc \rangle \;\triangleright^*\; \langle c,\, v : e,\, \sigma,\, s,\, pc + 1 \rangle$$

Proof. We evaluate the compilation function $\mathcal{CA}$ mapped to the `obtain` expression.

$$\begin{aligned} \langle \mathcal{CA}(E) : c,\, e,\, \sigma,\, s,\, pc \rangle &= \langle \mathcal{CA}(\texttt{obtain}(\&x, \phi)) : c,\, e,\, \sigma,\, s,\, pc \rangle \\ &= \langle \texttt{OBT}\ x,\, id_\phi,\, \phi : c,\, e,\, \sigma,\, s,\, pc \rangle && \text{(definition of } \mathcal{CA}) \\ &\triangleright \langle c,\, v : e,\, \sigma,\, s,\, pc + 1 \rangle && \text{(operational semantics of } \texttt{OBT}) \end{aligned}$$

If $\mathcal{W}$ fails and yields $\perp$, the high-level operational semantic rule $[\text{obt}^{\perp}_{\text{os}}]$ transitions the program state to $\hat{\sigma}$. Perfectly reflecting this, the abstract machine `OBT` instruction explicitly transitions to $\hat{\sigma}$ upon extraction failure. Therefore, the compilation strategy is correct. □

6.3 Transfinite Correctness of Divergent Programs

With the base cases for finite-step execution securely established for the `SelVeri LP` logic extensions, we must address the correctness of divergent (infinite looping or non-terminating) programs under this new paradigm.

The original mathematical proof for divergent programs in `SelVeri` relies on the *decomposition lemma* and transfinite induction (limit-ordinal induction) applied over repeating finite execution chunks. Because the `SelVeri LP` extensions ({ ϕ } and `obtain`) strictly evaluate in a finite number of discrete execution steps, and compile down to a finite, static sequence of `SelVerIR LP` instructions, they completely satisfy the prerequisite bounds of the decomposition lemma.

Consequently, the original transfinite induction proof remains fully valid without any modification. The resulting program states and stabilization limits after executing an infinite trace ω of a divergent `SelVeri LP` program and its compiled `SelVerIR LP` counterpart are guaranteed to be mathematically identical.

6.4 Determinism of SelVerIR LP

Having established the provable correctness of the compilation from `SelVeri LP` to `SelVerIR LP`, we can directly deduce the deterministic properties of the extended abstract machine.

Theorem: The `SelVerIR LP` abstract machine is semantically non-deterministic, perfectly mirroring the non-determinism of the high-level `SelVeri LP` language.

Proof. By the correctness of the compilation function $\mathcal{C}$, the operational semantics of the high-level language are strictly preserved in the intermediate representation. We can evaluate the determinism of the transition relation ($\triangleright$) for the two new abstract machine instructions:

- **Determinism of `VERIP`:** The `VERIP` instruction relies exclusively on the boolean evaluation $\mathcal{M}_{\texttt{Z3}}(\sigma, s) \models \mathcal{ZF}(\phi)$. As argued in the high-level determinism proofs, this FOL entailment yields a single, mathematically deterministic truth value (1, 0, or $\perp$). Thus, the execution of `VERIP` preserves a strict partial function transition.
- **Non-Determinism of `OBT`:** The `OBT` instruction directly invokes the high-level witness extraction function $\mathcal{W}(x, \phi, \sigma, s)$ to determine which value v is pushed to the stack e. Because $\mathcal{W}$ maps to a set of valid witnesses for a loosely constrained formula ϕ (e.g., $\exists\, x \in \mathbf{Int}.\ \&x > 0 \wedge \&x < 5$), the `OBT` instruction can legally transition a single abstract machine configuration into multiple distinct configurations (e.g., pushing 1, 2, 3, or 4 to the stack).

Because a single `SelVerIR LP` configuration containing an `OBT` instruction can branch into multiple mathematically valid subsequent IR-configurations, the intermediate transition relation ($\triangleright$) ceases to be a partial function. Therefore, the abstract machine inherits the exact non-deterministic nature of the high-level language, exactly as the compiler correctness proofs imply. □